Strategic Implementation Architecture for Offline Reinforcement Learning in Renewable Energy Communities

The Evolution of Energy Management Systems

The transition from centralized, fossil-fuel-dependent power grids toward decentralized, sustainable energy systems has catalyzed the emergence of Renewable Energy Communities (RECs). These communities represent a fundamental restructuring of energy distribution, transforming passive consumers into active prosumers who generate, consume, and store energy locally.[1] The integration of distributed energy resources (DERs), such as solar photovoltaic (PV) arrays, battery energy storage systems (BESS), and electric vehicles (EVs), offers profound environmental and economic advantages. However, it simultaneously introduces severe technical complexities. The stochastic nature of weather-dependent renewable generation, coupled with highly volatile human consumption patterns and unpredictable EV charging behaviors, creates persistent temporal mismatches between energy supply and demand.[1]

Historically, Energy Management Systems (EMS) have relied on classical control strategies, fuzzy logic controllers, and metaheuristic optimization algorithms to balance these supply-demand fluctuations.[1] While conventional methods ensure operational stability, they frequently fail to achieve absolute economic efficiency in complex, multi-objective environments. The subsequent adoption of online Reinforcement Learning (RL) marked a significant advancement, enabling autonomous agents to learn optimal control policies through continuous trial-and-error interaction with the environment.[1] Within the context of the OPEVA project and the EnergAIze platform, early iterations utilized algorithms such as the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) to optimize REC energy flows.[1]

Despite achieving reductions in operational costs, online RL paradigms exhibit critical limitations that hinder widespread real-world deployment. The fundamental reliance on continuous environmental interaction during the training phase introduces severe safety risks; an exploratory policy might execute destructive actions, such as violating thermal comfort boundaries or inducing excessive battery degradation.[1] Furthermore, relying purely on simulated environments introduces the persistent sim-to-real gap, where discrepancies between the modeled thermodynamics and actual physical dynamics cause the deployed policy to fail catastrophically.[1] Consequently, a paradigm shift toward Offline Reinforcement Learning (ORL) is mathematically and operationally imperative. By deriving optimal control strategies exclusively from static, historically collected datasets, ORL decouples the learning process from active environmental interaction, guaranteeing safety during training and

drastically improving sample efficiency.[1]

Theoretical Foundations and Systematic Synthesis

To successfully construct an architecture capable of inter-community and intra-community zero-shot generalization, the implementation must be deeply grounded in the theoretical foundations of offline reinforcement learning. The core challenge of ORL is extrapolation error, commonly referred to as distributional shift.[1] In a standard online off-policy algorithm (such as standard Q-learning or Deep Deterministic Policy Gradient), the agent continuously updates its target values by estimating the maximum possible future reward.[1] In an offline setting, the agent cannot explore the environment to verify these estimates. If the policy proposes an action that is absent from the historical dataset (an out-of-distribution or OOD action), the neural network approximation of the Q-function may erroneously assign it an artificially high value.[1] The agent will subsequently optimize its behavior toward these hallucinatory rewards, leading to catastrophic policy degradation during real-world deployment.[4]

The contemporary literature resolves the distributional shift problem through several distinct mathematical frameworks, which heavily inform the algorithmic selection for this implementation plan.

Constraint-Based and Regularization Frameworks

The first major approach involves explicit policy constraints, aiming to confine the learned policy strictly within the support of the behavioral dataset. Algorithms such as Batch-Constrained Q-learning (BCQ) utilize a generative model, typically a Variational Autoencoder (VAE), to learn the distribution of the historical data.[1] During policy optimization, BCQ samples multiple actions from the VAE and selects the one with the highest estimated Q-value, thereby guaranteeing that the agent only executes actions statistically similar to those previously observed.[1] While BCQ has demonstrated near-optimal performance in hybrid electric vehicle energy management, its reliance on a high-fidelity generative model introduces significant computational overhead and tuning complexity.[1]

An alternative approach centers on value function regularization. Conservative Q-Learning (CQL) directly addresses overestimation by appending a penalty term to the Bellman error objective during the training of the critic network.[1] This penalty actively pushes down the Q-values of actions that are not present in the dataset, while pushing up the values of actions that are present.[1] CQL establishes a rigorous theoretical lower bound on the true value function, creating a mathematically safe baseline.[1] In multi-zone building cluster environments, CQL has successfully coordinated HVAC systems and battery storage, achieving significant reductions in total energy costs.[1] However, the intense conservatism of CQL frequently limits its ability to generalize to novel building topologies, as the severe penalties restrict the agent's adaptability in unseen states.[1]

In-Sample Learning and Minimalist Approaches

To achieve true zero-shot generalization, the architecture must balance the prevention of extrapolation error with the flexibility necessary to adapt to unseen environmental dynamics. This necessitates a shift toward minimalist regularization and in-sample dynamic programming.

The Twin Delayed Deep Deterministic Policy Gradient with Behavior Cloning (TD3+BC) algorithm represents the optimal baseline prototype for this architecture.[1] Rather than altering the underlying structure of the critic networks or training secondary generative models, TD3+BC modifies the deterministic policy gradient update of the actor network by appending a straightforward behavioral cloning regularization term.[1] The objective function balances the maximization of the Q-value against the minimization of the mean squared error between the agent's proposed action and the actual action recorded in the dataset.[1] To ensure stability across diverse state spaces with varying reward magnitudes, the regularization weight is normalized by the absolute mean of the Q-values across the mini-batch.[1] The extreme simplicity of TD3+BC ensures rapid convergence and provides a highly stable benchmark for continuous control tasks within the energy community.[1]

Despite the utility of TD3+BC, Implicit Q-Learning (IQL) stands as the superior mathematical engine for achieving inter-community transfer.[3] Traditional offline algorithms inevitably query the Q-function with actions generated by the current, unverified policy. IQL elegantly circumvents this fatal flaw through the application of expectile regression.[3] Instead of directly estimating the value of the policy's actions, IQL treats the state-value function as a random variable determined by the action distribution.[5] By fitting an asymmetric squared error loss, the algorithm approximates the upper expectile of this random variable.[4] This effectively estimates the maximum Q-value over the actions that actually exist within the dataset, completely eliminating the necessity of evaluating out-of-distribution actions.[4]

Following the optimization of the value function via these purely in-sample SARSA-style temporal difference backups, the final policy is extracted using advantage-weighted behavioral cloning.[3] Because the entire optimization loop relies exclusively on historical data points, IQL constructs a highly generalizable value landscape.[5] This capacity for multi-step dynamic programming allows the agent to stitch together suboptimal historical trajectories into a near-optimal policy that remains robust when transferred zero-shot to entirely novel climatic conditions or building structures.[1]

Dataset Architecture and State-Space Engineering

The operational success of Implicit Q-Learning relies fundamentally on the structural integrity, granularity, and dimensionality of the historical data.[1] The project leverages the CityLearn simulation ecosystem, which provides deeply integrated datasets modeling diverse prosumer profiles, including residential dwellings, office spaces, and industrial facilities.[1] These datasets

are formatted as JSON schemas within the datasets/ repository directory, ensuring complete parity between local development and containerized execution.[1]

Unlike siloed datasets that merely record isolated energy transactions, the CityLearn framework captures the holistic, interdependent dynamics of a decentralized grid.[1] To facilitate robust offline training, the raw historical logs must be transformed into a standardized Markov Decision Process (MDP) format. This transformation is governed by the observation encoders defined within configs/encoders/default.json, which map the raw simulator telemetry into the normalized state vector required by the neural networks.[1]

The engineered state space must comprehensively capture temporal context, thermodynamic forces, local asset status, and external market pressures. The specific vector components and their strategic rationale are detailed below.

Feature Classification	Specific Data Signals	Preprocessing and Normalization Strategy	Architectural Function in Energy Management
Temporal Context	month, day_type, hour_of_day	Cyclical encoding utilizing sine and cosine transformations.	Empowers the agent to capture diurnal patterns and seasonal variations, critical for anticipating cyclical solar irradiance and routine human occupancy behaviors.[1]
Meteorological Dynamics	outdoor_dry_bulb_temp, relative_humidity, direct_solar_irradiance	Min-Max scaling constrained by historical climatic extremes.	Drives the predictive dynamics of the building's thermal envelope, dictating future HVAC cooling and heating load requirements.[1]
Asset Telemetry	non_shiftable_load, cooling_demand, dhw_demand,	Z-score normalization to ensure variance	Defines the immediate, inflexible energy

	solar_generation	stability.	deficit or surplus of the dwelling, forming the baseline from which flexible optimization occurs.[1]
EV Flexibility States	current_soc, required_soc_departure, estimated_arrival_soc	Bounded continuous ratio (0.0 to 1.0).	Quantifies the available mobility constraints, defining the exact temporal window during which Vehicle-to-Grid (V2G) discharging is permissible without stranding the user.[1]
Market and Environmental Signals	electricity_pricing, pricing_predicted_6h, pricing_predicted_12h, carbon_intensity	Moving average normalization relative to trailing 24-hour windows.	Provides the external economic and ecological incentives that shape the reward function, enabling proactive energy arbitrage.[1]

The action space is equally critical and must be engineered as a continuous, bounded vector ranging from -1.0 to 1.0. A negative scalar dictates energy storage (importing power from the grid to charge local batteries or EVs), while a positive scalar dictates energy discharging (exporting power back to the grid to capture market revenue or alleviate community peak demand).[1]

However, standardizing the dynamic telemetry is insufficient for zero-shot generalization.[1] If an agent is trained exclusively on the telemetry of a massive industrial facility, it will develop a policy that attempts to discharge massive quantities of energy during high-price events. If that exact policy is deployed zero-shot onto a small residential apartment, the agent will attempt to request discharge rates that vastly exceed the physical limitations of the apartment's inverter, leading to immediate constraint violations.[1]

To prevent this architectural failure, the state space must be augmented with static, contextual fingerprints. These features include parameters such as the building's nominal_pv_capacity, battery_capacity, and envelope_thermal_mass.[1] By embedding these physical constraints

directly into the observation vector, the neural network learns to interpret its continuous action output (-1.0 to 1.0) as a relative percentage of the building's specific maximum capacity, rather than an absolute kilowatt-hour request. This contextualization is the mathematical linchpin that allows a single global policy to safely transition between wildly divergent architectural topologies.[1]

The Strategic Imperative of Data Augmentation

The inherent limitation of offline reinforcement learning is that the dataset represents a finite, historical snapshot of the environment. The CityLearn 2022 Challenge dataset, while comprehensive, is inherently biased by the specific climatic conditions of Fontana, California, and the specific behavioral routing of the default heuristic controllers.[1] If the behavioral policy that generated the data never explored certain regions of the state space—such as extreme heatwaves causing unprecedented cooling demands, or anomalous EV charging behaviors—the offline agent cannot learn optimal recovery strategies for those states.[1]

When transitioning to inter-community transfer, the target community will inevitably present states outside the original training distribution.[1] Therefore, data augmentation is not merely an optional performance enhancement; it is a critical prerequisite for immunizing the policy against structural overfitting and ensuring robust zero-shot deployment.[7]

The implementation architecture must deploy sophisticated, time-aware data augmentation pipelines to artificially expand the support of the behavioral policy.[9] Traditional data augmentation techniques utilized in computer vision, such as random masking or simple noise injection, are destructive in energy timeseries analysis because they sever the thermodynamic and physical continuity of the sequence.[8] Instead, the augmentation strategy must focus on realistic non-stationarity.

The framework must integrate a Temporal Distance-Aware Transition Augmentation (TempDATA) or Time-series Generative Adversarial Network (TimeGAN) module.[8] This generative architecture learns the underlying autoregressive transition dynamics of the grid. Once trained, it can synthesize highly realistic, multi-step trajectories that preserve the strict physical correlations between solar irradiance, outdoor temperature, and resultant energy consumption.[8]

The generative pipeline must systematically inject specific forms of entropy into the replay buffer. First, meteorological augmentation should synthesize extreme weather anomalies. By generating synthetic trajectories that simulate extended periods of high heat and low solar irradiance, the agent is forced to learn conservative energy dispatching strategies, ensuring it maintains sufficient battery reserves to satisfy critical cooling loads.[1] Second, behavioral entropy must be injected into the Electric Vehicle mobility patterns. The augmentation module must generate edge-case scenarios involving highly erratic EV departure times and anomalous target states-of-charge.[1] By exposing the IQL and TD3+BC algorithms to these synthetic,

high-variance scenarios during the offline training phase, the resulting value functions become significantly more resilient. This expanded dataset diversity directly minimizes the generalization gap, guaranteeing that the policy will not collapse when encountering unforeseen operational disturbances in the target community.[1]

Architecting Zero-Shot Generalization Mechanisms

The overarching scientific ambition of this implementation is the realization of zero-shot generalization. The deployment architecture must facilitate both intra-community and inter-community transfer without requiring any online fine-tuning or secondary gradient updates.[1] This capability drastically reduces the computational overhead and integration latency associated with onboarding new prosumers to an existing smart grid.

Intra-Community Generalization and Parameter Sharing

Intra-community generalization tests the agent's ability to optimize energy flows in unseen buildings that reside within the same overarching microgrid as the training set.[1] The standard experimental protocol involves training the agent on a subset of the community (e.g., Buildings 1 through 10) and deploying it zero-shot onto the remaining structures (Buildings 11 through 17).[1]

To achieve this, the architecture must abandon the concept of independent learners in favor of a Centralized Training with Decentralized Execution (CTDE) paradigm heavily reliant on parameter sharing.[1] In a traditional multi-agent setup, initializing a distinct neural network for every individual building scales poorly and fragments the learning process. By employing parameter sharing, a single, highly expressive global actor network and global critic network are instantiated. The historical trajectories from Buildings 1 through 10 are aggregated into a massive, unified offline replay buffer.[1]

The global network learns generalizable thermodynamic principles and market arbitrage strategies from this collective experience. Because the state vector includes the static contextual fingerprints detailed previously, the global policy dynamically adjusts its continuous output based on the specific parameters of the building it is currently controlling.[1] When Building 11 is integrated into the grid, the global actor ingests its unique nominal_pv_capacity and non_shiftable_load profile, instantly outputting highly tailored optimization commands.[1]

However, to guarantee absolute safety during this zero-shot transfer, the system must employ Cluster-Based Policy Mapping supported by a Runtime Shielding Mechanism.[1] Before the global policy is permitted to dispatch commands to an unseen building, an automated clustering algorithm (such as Dynamic Time Warping applied to historical base load consumption) must verify that the new building's energy profile statistically aligns with the centroid boundaries of the training distribution.[1] If the unseen building exhibits a fundamentally anomalous profile, the Runtime Shielding Mechanism must intercept the RL agent's commands.

The system must seamlessly transition control to the deterministic RuleBasedPolicy fallback.[1] This hybrid architecture ensures that the pursuit of algorithmic generalization never compromises the physical integrity or safety of the community infrastructure.[1]

Inter-Community Transfer and Dimensional Reconciliation

Inter-community generalization represents a significantly more complex computational hurdle. This protocol requires a policy trained exclusively on one distinct geographical and structural dataset (e.g., the CityLearn Challenge 2022 dataset) to operate zero-shot on an entirely different grid topology (e.g., the three-phase electrical service demonstration dataset).[1] The profound difficulty here lies not just in differing weather patterns, but in structural input space mismatches.[1]

If the target community lacks specific infrastructure—for example, if it possesses residential batteries but no electric vehicle charging stations—the standard observation vector will encounter missing features, causing the neural network to fail catastrophically during forward propagation.[1] To resolve this, the utils/wrapper_citylearn.py component must be engineered to function as a Universal State/Action Wrapper.[1]

This universal wrapper must dynamically construct the observation space based on the highest-dimensional schema available.[1] When the policy is deployed to a community lacking specific DERs, the wrapper must execute zero-imputation or mean-imputation for the missing features, effectively masking out the irrelevant dimensions without altering the expected shape of the input tensor.[1] The architecture relies heavily on the Implicit Q-Learning algorithm's inherent robustness in this phase. Because IQL's value function was optimized purely on in-sample trajectories and its policy extracted via advantage-weighted behavioral cloning, it is uniquely resistant to the catastrophic divergence typically triggered by shifting feature distributions.[4] The success of this inter-community transfer will be rigorously quantified by the Generalization Gap, defined as the ratio of the target community performance divided by the source community performance, with a strict architectural target of exceeding 0.8.[1]

Repository Integration and Software Engineering Contracts

The theoretical design of the offline algorithms and generalization wrappers must be seamlessly integrated into the specific software engineering contracts of the EnergAIze repository.[1] The repository is structurally designed to abstract orchestration overhead away from the developer, allowing for intense focus on mathematical optimization.[1]

The primary execution flow is governed by the run_experiment.py entrypoint, which supports local, containerized (Docker), and High-Performance Computing (Deucalion/SIF) deployments.[1] Upon execution, the framework immediately consults the utils/config_schema.py to validate

the structural integrity of the experiment's YAML configuration using Pydantic models.[1] This validation ensures that hyperparameters critical to the offline algorithms—such as the IQL expectile threshold (τ), the TD3+BC regularization scalar (α), and the batch size—are correctly typed and bounded before allocating computational resources.[1]

To integrate the new algorithms, the IQLAgent and TD3BCAgent classes must be instantiated within the algorithms/agents/ directory and must strictly inherit from the BaseAgent class.[1] The developer must fulfill the mandatory lifecycle methods dictated by this base contract [1]:

1. **predict(observations, deterministic):** This method receives the normalized, context-enriched state vectors from the Wrapper_CityLearn module.[1] Because the environment is offline, the agent must output deterministic actions, executing the argmax of the value function without the injection of Gaussian exploration noise.[1]
2. **update(...):** The core optimization loop. In traditional online settings, this method ingests streaming environmental transitions. For the offline architecture, the wrapper must provide mini-batches sampled directly from the synthesized, augmented historical replay buffer.[1] The method must rigorously respect the system's keyword-only scheduling flags, such as update_target_step and global_learning_step, to ensure Polyak averaging of the target networks and loss logging remain perfectly synchronized.[1]
3. **export_artifacts(output_dir, context):** The most critical function for deployment.[1] The ultimate goal of the offline training pipeline is to produce a portable, highly optimized controller. This method must freeze the PyTorch or TensorFlow computational graphs and serialize them into the ONNX (Open Neural Network Exchange) format, strictly utilizing the pinned opset version 13.[1]

The export_artifacts method must also return a structured dictionary that populates the artifact_manifest.json.[1] This manifest is the definitive contract for the downstream inference serving API. It must comprehensively document the specific configs/encoders/default.json rules utilized during training, the precise action bounds (-1.0 to 1.0), and the reward formulations.[1] Any discrepancy between the preprocessing logic documented in the manifest and the actual operations expected by the ONNX actor will result in catastrophic failure during deployment on REC edge devices.[1] Furthermore, the agent must be explicitly registered within the ALGORITHM_REGISTRY located in algorithms/registry.py; bypassing this step will result in a fatal instantiation error by the runner.[1]

Exhaustive Implementation Roadmap

To execute this complex architecture efficiently, the integration process must be structured into a highly granular, milestone-driven execution roadmap spanning the project lifecycle.

Phase 1: Architectural Foundation and Diagnostic Analysis

The initial phase establishes the data pipeline and diagnostic baseline.

- **Actionable Task 1.1:** Develop the OfflineBaseAgent class, extending the core BaseAgent to standardize replay buffer ingestion and batch processing mechanics.[1]
- **Actionable Task 1.2:** Conduct exhaustive Dataset Diagnostics. Extract the citylearn_challenge_2022 historical CSVs. Execute dimensionality reduction (PCA or t-SNE) on the state-action pairs to visually map the support manifold of the underlying behavioral policy.[1] Identify sparse regions within the distribution that require targeted augmentation.
- **Actionable Task 1.3:** Construct the contextual fingerprinting encoders within the Wrapper_CityLearn module, appending static parameters like nominal_pv_capacity to the dynamic telemetry vector.[1]

Phase 2: Generative Augmentation and Prototype Baselining

This phase stabilizes the optimization environment and establishes the primary mathematical baselines.

- **Actionable Task 2.1:** Implement the Temporal Distance-Aware Transition Augmentation (TempDATA) pipeline. Synthesize auxiliary historical logs injecting extreme meteorological conditions and EV behavioral entropy, merging them into the unified replay buffer.[8]
- **Actionable Task 2.2:** Develop and register the **TD3+BC** agent.[1] Configure the config templates in configs/templates/offline_td3bc.yaml.
- **Actionable Task 2.3:** Execute initial training runs via run_experiment.py. Tune the regularization scalar α. Verify that the actor loss curves converge rapidly and that the resulting KPIs strictly match or slightly exceed the baseline efficiency of the RuleBasedPolicy.[1]

Phase 3: Advanced IQL Optimization and Offline Evaluation

The deployment of the core scientific contribution and safety verifications.

- **Actionable Task 3.1:** Implement the **Implicit Q-Learning (IQL)** algorithm. Structure the update method to perform precise expectile regression on the state-value network, ensuring no out-of-distribution actions are ever queried during the Bellman updates.[3]
- **Actionable Task 3.2:** Execute rigorous hyperparameter sweeps on the expectile threshold (τ) and the inverse temperature parameter (β) used during advantage-weighted behavioral cloning.[1]
- **Actionable Task 3.3:** Implement Offline Policy Evaluation (OPE) algorithms, such as Fitted Q-Evaluation (FQE). Mathematically verify the expected return of the IQL policy strictly from the dataset before ever exposing it to the live CityLearn simulator.[1]

Phase 4: Zero-Shot Generalization and EV Extension

Testing the boundaries of the parameter-sharing architecture.

- **Actionable Task 4.1:** Execute the Intra-Community Generalization protocol. Train the global IQL policy using data from Buildings 1-10. Deploy the frozen policy onto Buildings 11-17.[1] Track the cost reduction and peak shaving KPIs via MLflow.[1]
- **Actionable Task 4.2:** Integrate the Universal State Wrapper to handle EV charger states. Activate the Runtime Shielding Mechanism, guaranteeing that any target building with a load profile failing the Dynamic Time Warping cluster check immediately reverts to the RBC fallback.[1]
- **Actionable Task 4.3:** Quantify the Generalization Gap. Perform rigorous statistical analysis utilizing paired t-tests across 3 to 5 unique random seeds to verify that performance degradation on unseen buildings remains within acceptable bounds (targeting a ratio > 0.8).[1]

Phase 5: Inter-Community Transfer and Ecosystem Deployment

The ultimate stress test and serialization of the optimal policy.

- **Actionable Task 5.1:** Execute the Inter-Community Transfer protocol. Deploy the IQL policy, trained purely on the 2022 challenge data, zero-shot onto the three-phase electrical service demo environment.[1] Monitor the Universal Wrapper's ability to seamlessly mean-impute missing observation dimensions.[1]
- **Actionable Task 5.2:** Conduct comprehensive ablation studies. Systematically mask pricing signals and carbon intensity indicators to evaluate the policy's resilience to signal loss and sensor failure.[1]
- **Actionable Task 5.3:** Validate the export_artifacts pipeline. Confirm the ONNX actor models are generated utilizing opset 13, and rigorously verify that the artifact_manifest.json perfectly mirrors the deployment configuration requirements of the downstream inference API.[1]

By systematically executing this meticulously engineered roadmap, the implementation will circumvent the fatal flaws of traditional online reinforcement learning. The strategic fusion of synthetic data augmentation, contextual parameter-sharing, and the mathematical safety of Implicit Q-Learning guarantees the creation of a highly scalable, robust, and universally deployable energy management policy capable of redefining operational efficiency in Renewable Energy Communities.

Trabalhos citados

1. thesis.pdf
2. Coordinated Energy Management for Inter-Community Imbalance Minimization - Matsuyama Lab, acesso a março 21, 2026, https://vision.kuee.kyoto-u.ac.jp/japanese/happyou/pdf/Rodrigo_RENE_2015.pdf
3. [2110.06169] Offline Reinforcement Learning with Implicit Q-Learning - arXiv,

acesso a março 21, 2026, https://arxiv.org/abs/2110.06169
4. Offline Reinforcement Learning with Implicit Q-Learning, acesso a março 21, 2026, https://offline-rl-neurips.github.io/2021/pdf/24.pdf
5. Offline Reinforcement Learning with Implicit Q-Learning - OpenReview, acesso a março 21, 2026, https://openreview.net/forum?id=68n2s9ZJWF8
6. Deep reinforcement learning with online data augmentation to improve sample efficiency for intelligent HVAC control (Conference) | OSTI.GOV, acesso a março 21, 2026, https://www.osti.gov/biblio/1907234
7. Small Dataset, Big Gains: Enhancing Reinforcement Learning by Offline Pre-Training with Model-Based Augmentation - MDPI, acesso a março 21, 2026, https://www.mdpi.com/2813-0324/9/1/4
8. Data Augmentation for Reinforcement Learning | ORNL, acesso a março 21, 2026, https://www.ornl.gov/research-highlight/data-augmentation-reinforcement-learning
9. Forecasting in Offline Reinforcement Learning for Non-stationary Environments - arXiv.org, acesso a março 21, 2026, https://arxiv.org/html/2512.01987v3
10. Model-Based Offline Reinforcement Learning with Adversarial Data Augmentation - arXiv, acesso a março 21, 2026, https://arxiv.org/html/2503.20285
11. Time Series Data Augmentation for Energy Consumption Data Based on Improved TimeGAN - MDPI, acesso a março 21, 2026, https://www.mdpi.com/1424-8220/25/2/493
12. ICML Poster Temporal Distance-aware Transition Augmentation for Offline Model-based Reinforcement Learning, acesso a março 21, 2026, https://icml.cc/virtual/2025/poster/44612
13. A novel twin time series network for building energy consumption predicting - PMC, acesso a março 21, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12200849/
14. CityLearn — CityLearn 2.5.0 documentation, acesso a março 21, 2026, https://www.citylearn.net/
15. NeurIPS Poster Towards Robust Zero-Shot Reinforcement Learning, acesso a março 21, 2026, https://neurips.cc/virtual/2025/poster/115398